

MAKING VISIONS WORK.

Co-Creating Digital Businesses, Digital Experiences & Products



MILESTONE PREVIEW

Quan Phan | 24/08/2025

Agenda

1. Top-down approach, Database Analysis & Design
2. Critical Thinking & Writing SQL Query
3. Writing unit test & Learn Clean Code Principle
4. Writing Custom Framework & Outdoor challenge
5. Summary

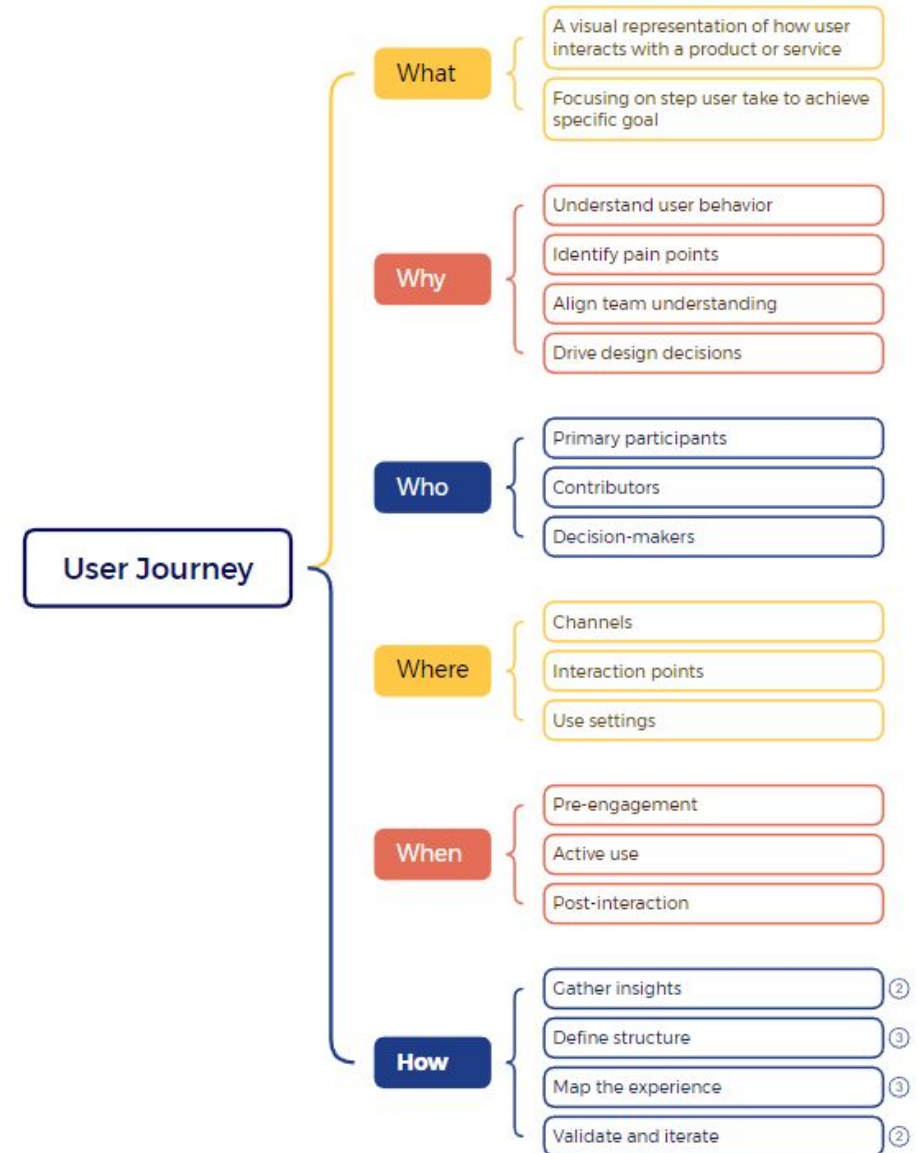




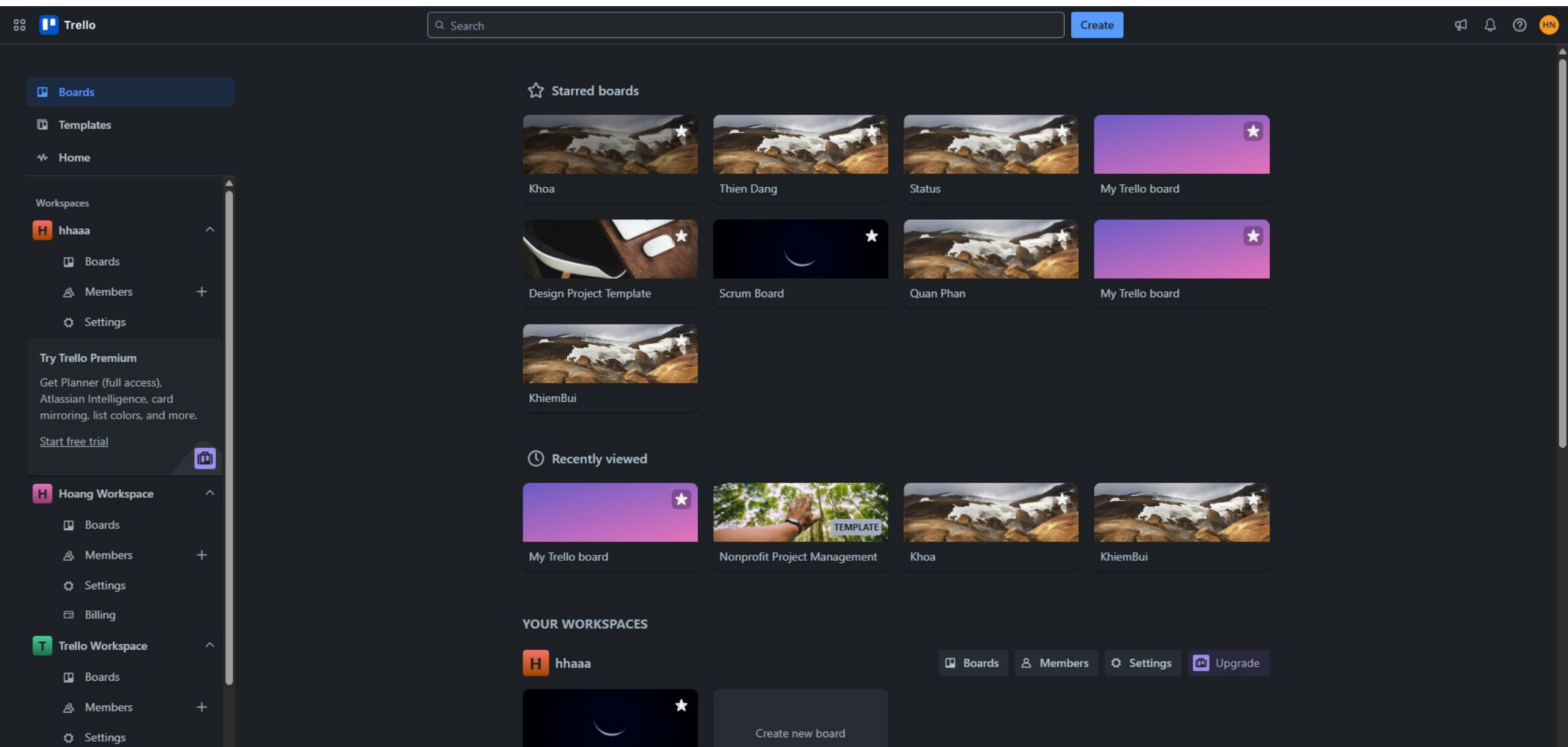
MILESTONE 1: TOP-DOWN APPROACH, DATABASE ANALYSIS & DESIGN

LEARN TOP-DOWN APPROACH

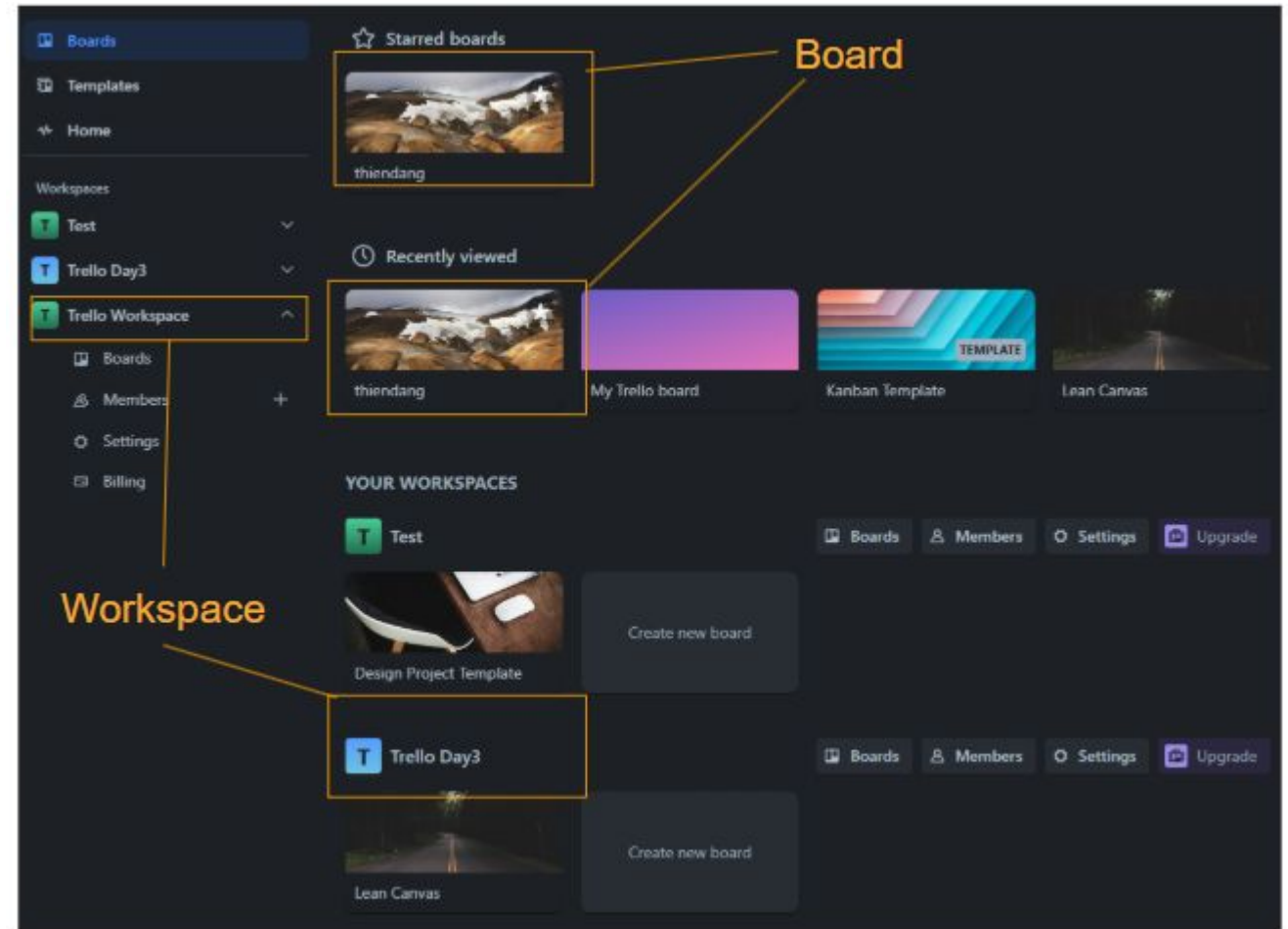
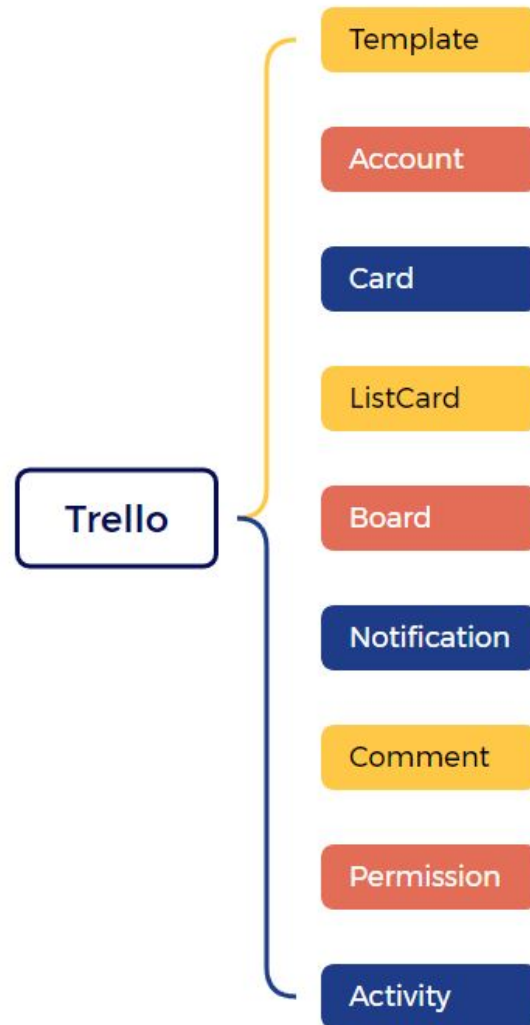
- Learn how to use mind map to learn new concept by 5W 1H
- Learn how to break a big concept to many smaller concepts
- Learn how to start from the overall picture before focusing on details
- Learn how to organize knowledge hierarchically from general to specific



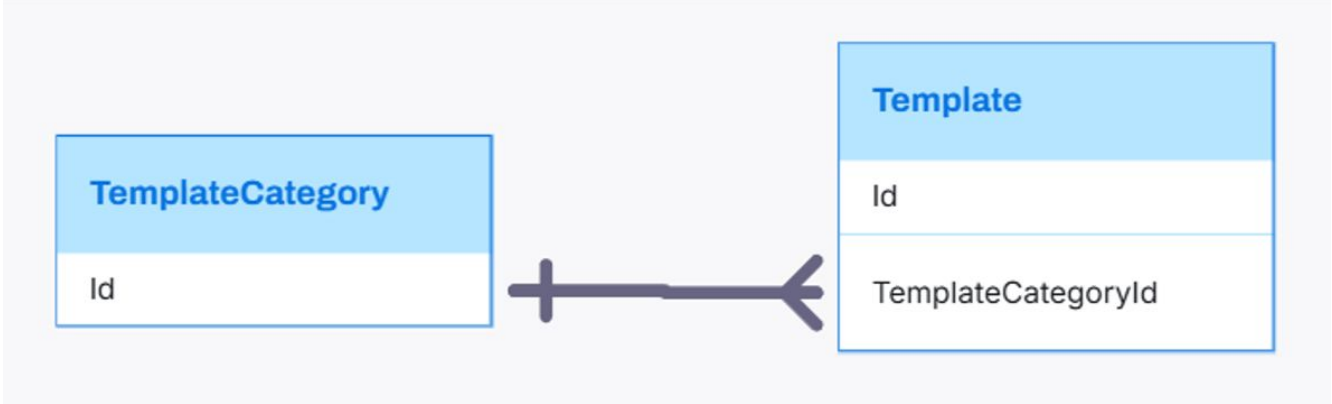
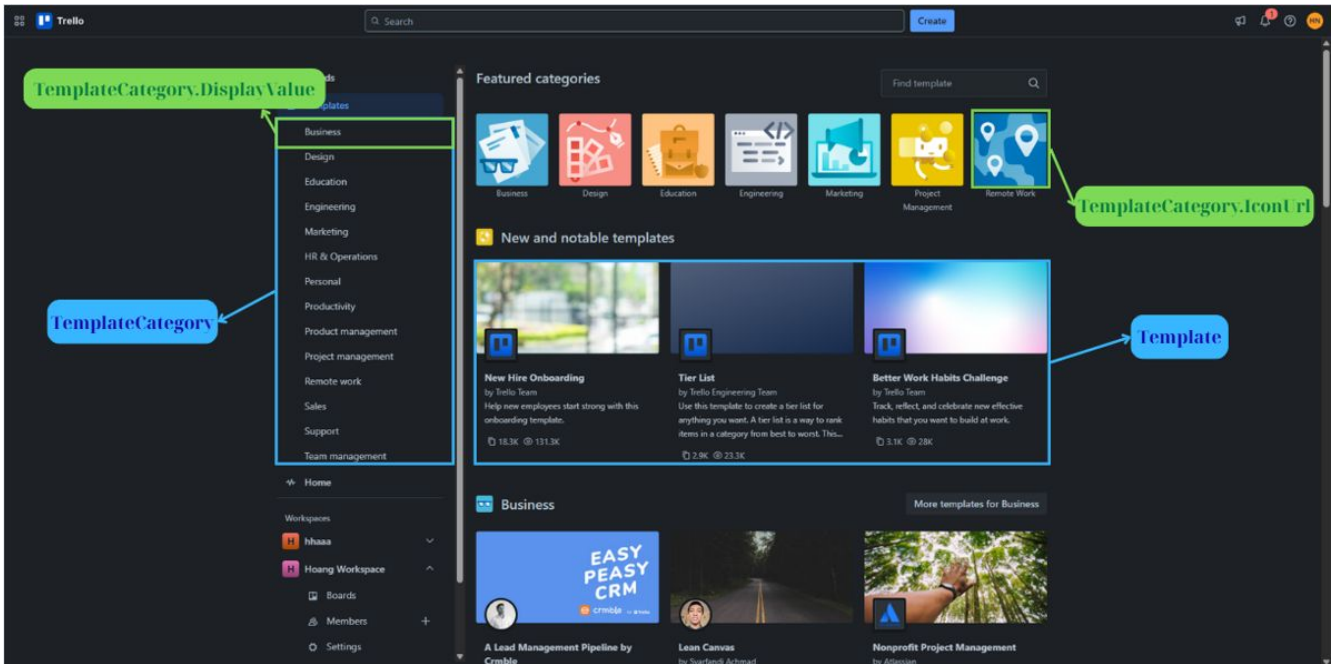
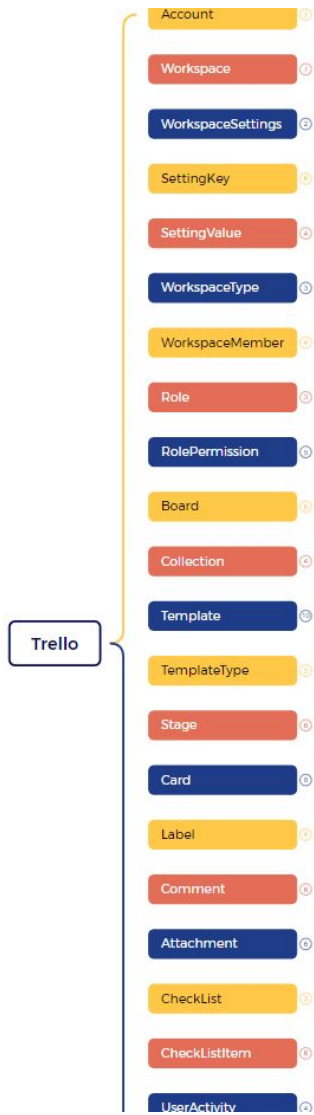
DESIGN DATABASE FOR TRELLO



FIRST VERSION OF LIST ENTITY FOR TRELLO



VERSION AFTER RECEIVING FEEDBACK





MLESTONE 2: Learn Critical Thinking & Write SQL Query

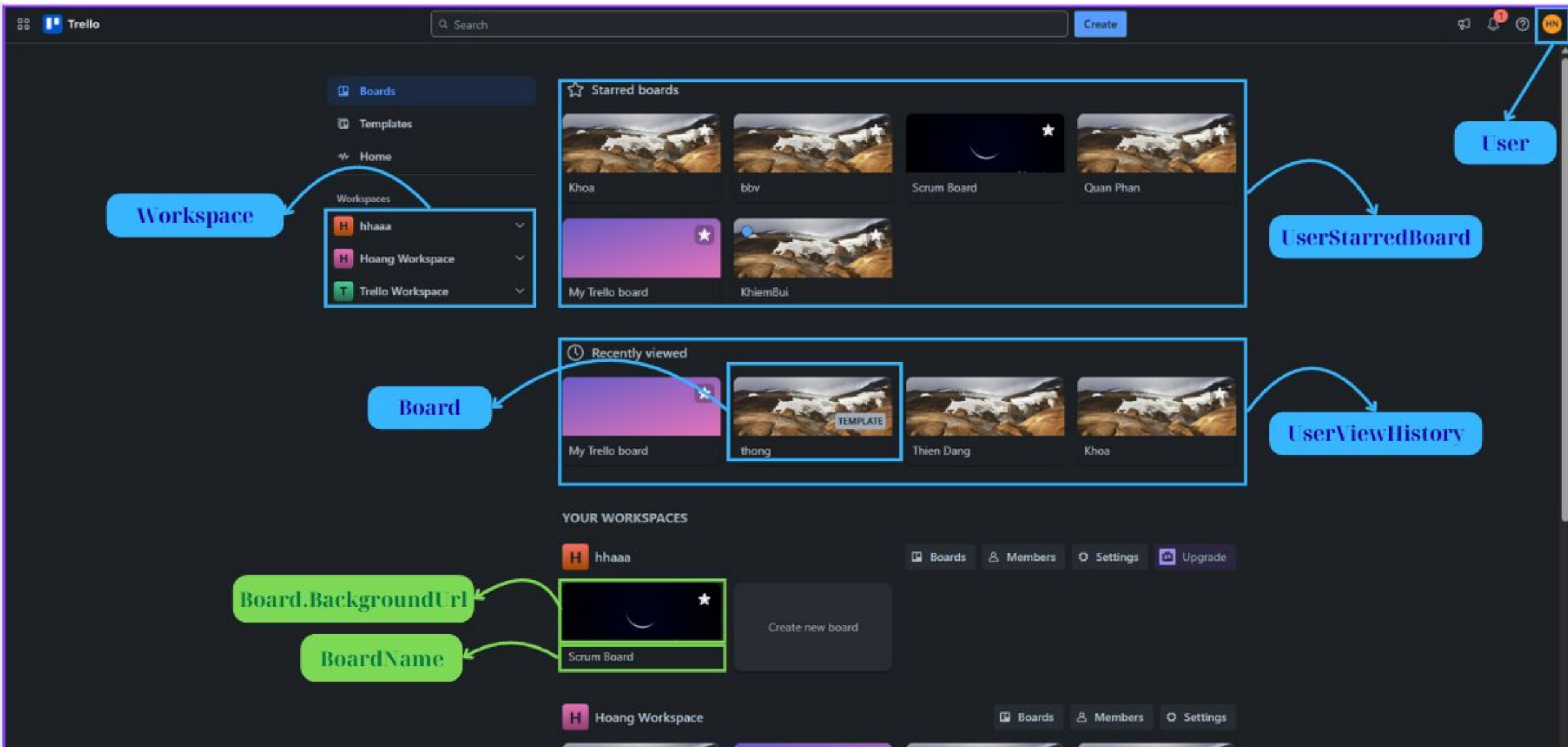
CRITICAL THINKING – BIAS & FALLACY

BIAS - THIÊN KIẾN	FALLACY - NGỤY BIỆN
Stereotype Bias (Định kiến nhóm): <i>Using societal stereotypes to judge a group of people</i>	Appeal to Emotion (Dựa vào cảm xúc): <i>Using emotion to persuade instead of logical reasoning</i>
Halo Effect (Hiệu ứng hào quang): <i>Using one prominent characteristic to form an overall positive judgment</i>	Ad Hominem (Tấn công cá nhân): <i>Attacking the person instead of evaluating the merit of their actions/record</i>
Confirmation Bias (Thiên kiến xác nhận): <i>Paying attention only to information that confirms existing beliefs, while ignoring contradictory evidence</i>	Bandwagon Fallacy (Theo số đông): <i>Assuming something is true because many people believe it</i>
Availability Bias (Thiên kiến sẵn có): <i>Making decisions based on easily recalled, recent examples or information</i>	False Dilemma (Lưỡng nan giả tạo): <i>Presenting only 2 extreme options, when other solutions might exist</i>
Anchoring Bias (Thiên kiến mỏ neo): <i>Relying too heavily on the first piece of information received (the anchor) when making a decision</i>	Slippery Slope (Dốc trơn trượt): <i>Claiming one action will inevitably lead to a series of negative, unavoidable consequences</i>
In-group Bias (Thiên kiến nhóm trong): <i>Preferring individuals who share similar backgrounds, professions, or circumstances, etc.</i>	Appeal to Authority (Dựa vào quyền lực): <i>Assuming someone is correct because of their title, regardless of relevant expertise</i>
	Hasty Generalization (Khái quát vội vàng): <i>Drawing a broad conclusion from a few isolated examples</i>

ENHANCING CRITICAL THINKING

- **Ask questions:** Always ask "Why?", "How?", "What's the evidence?".
- **Seek diverse information:** Don't rely on a single source.
- **Challenge your own beliefs:** Be willing to change your mind if presented with convincing evidence.
- **Understand fallacies and biases:** Recognize them in others and in yourself.
- **Focus on data and evidence:** Don't let emotions or prejudices dictate your judgment.

Write SQL Query By Screen



BEFORE AND AFTER RECEIVING FEEDBACK



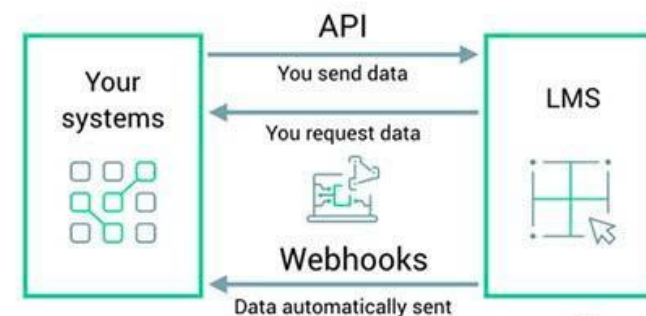
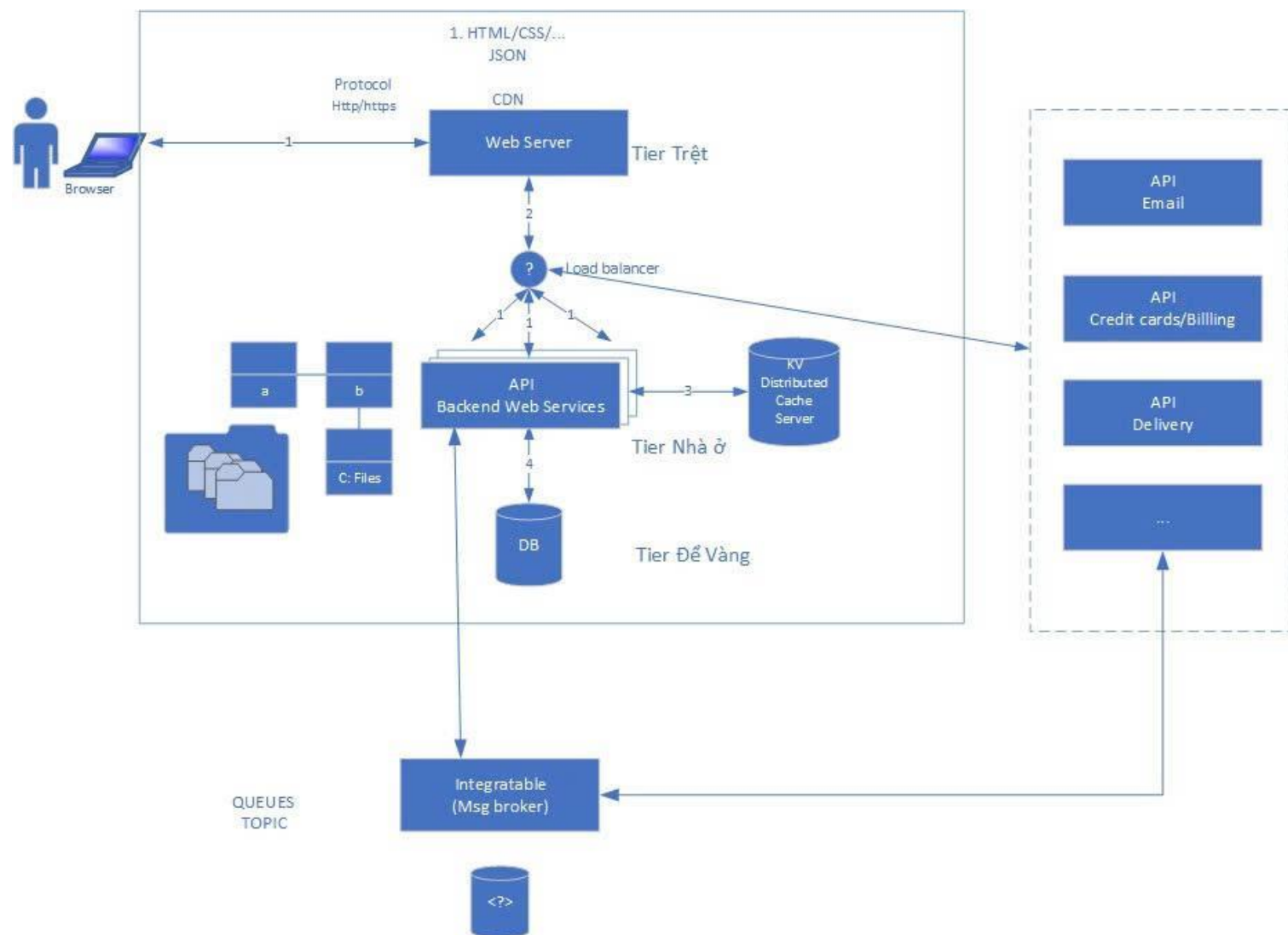
```
-- Query Starred Board By User
select b.Id,b.BoardName,b.BackgroundImage
from UserStarredBoard usb
join Board b on b.Id=usb.BoardId
where usb.UserId=1
order by usb.CreatedAt desc
```

```
--Get User Infor by Email
select usr.FullName,usr.PictureUrl,usr.Email
from Users usr
where usr.Email=''
```

```
-- Query Starred Board By User
SELECT
    b.Id,
    b.BoardName,
    b.BackgroundImage
FROM
    UserStarredBoard usb
    JOIN Board b ON b.Id=usb.BoardId
WHERE usb.UserId=1
ORDER BY usb.CreatedAt DESC
```

```
--Get User Infor by Email
SELECT
    usr.FullName,
    usr.PictureUrl,
    usr.Email
FROM
    Users usr
WHERE
    usr.Email = ''
```

LEARN THREE-TIER ARCHITECTURE





STONE 3: Writing Unit Test & Learn Clean Code Principle

WRITING UNIT TEST FOR TRELLO



```
@Test  👤 Quan Phan
void getSettingValueByKeyNameAndWorkspaceId(){
    //Arrange
    String keyName = "visibility";
    int workspaceId = 1;
    //Act
    WorkspaceSettingValueResponseDTO result = workspaceController.getSettingValueByKeyNameWorkspaceId(keyName
    //Assert
    assertNotNull(result);
}

@Test  👤 Quan Phan
void getMemberWorkspace(){
    //Arrange
    int workspaceId=1;
    //Act
    MemberWorkspaceResponseDTO result = workspaceController.getMembersByWorkspace(workspaceId);
    //Assert
    assertNotNull(result);
}
```

REFACTORING PROJECT UNIT TEST



refactoring UnitTest project and add customDI project

 phmquan committed last week · ✓ 1 / 1

97a769f  

refactoring project and implement H2 database

 phmquan committed 2 weeks ago · ✓ 1 / 1

3317a8e  

Refactoring Unit Test project and write FavoritedBoard test

 phmquan committed 2 weeks ago · ✓ 1 / 1

da6b999  

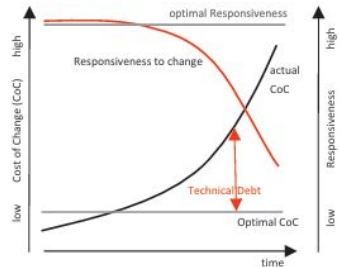
LEARN CLEAN CODE



Clean Code Cheat Sheet

Why Clean Code

Code is clean if it can be understood easily – by everyone on the team. With understandability comes readability, changeability, extensibility and maintainability. All the things needed to keep a project going over a long time without accumulating up a large amount of technical debt.



Writing clean code from the start in a project is an investment in keeping the cost of change as constant as possible throughout the lifecycle of a software product. Therefore, the initial cost of change is a bit higher when writing clean code (grey line) than quick and dirty programming (black line), but is paid back quite soon. Especially if you keep in mind that most of the cost has to be paid during maintenance of the software. Unclean code results in technical debt that increases over time if not refactored into clean code. There are other reasons leading to Technical Debt such as bad processes and lack of documentation, but unclean code is a major driver. As a result, your ability to respond to changes is reduced (red line).

In Clean Code, Bugs Cannot Hide

Most software defects are introduced when changing existing code. The reason behind this is that the developer changing the code cannot fully grasp the effects of the changes made. Clean code minimises the risk of introducing defects by making the code as easy to understand as possible.

Principles

Loose Coupling

Two classes, components or modules are coupled when at least one of them uses the other. The less these items know about each other, the looser they are coupled.

A component that is only loosely coupled to its environment can be more easily changed or replaced than a strongly coupled component.

High Cohesion

Cohesion is the degree to which elements of a whole belong together. Methods and fields in a single class and classes of a component should have high cohesion. High cohesion in classes and components results in simpler, more easily understandable code structure and design.

Change is Local

When a software system has to be maintained, extended and changed for a long time, keeping change local reduces involved costs and risks. Keeping change local means that there are boundaries in the design which changes do not cross.

It is Easy to Remove

We normally build software by adding, extending or changing features. However, removing elements is important so that the overall design can be kept as simple as possible. When a block gets too complicated, it has to be removed and replaced with one or more simpler blocks.

Mind-sized Components

Break your system down into components that are of a size you can grasp within your mind so that you can predict consequences of changes easily (dependencies, control flow, ...).

Smells

Rigidity

The software is difficult to change. A small change causes a cascade of subsequent changes.

Fragility

The software breaks in many places due to a single change.

Immobility

You cannot reuse parts of the code in other projects because of involved risks and high effort.

Viscosity of Design

Taking a shortcut and introducing technical debt requires less effort than doing it right.

Viscosity of Environment

Building, testing and other tasks take a long time. Therefore, these activities are not executed properly by everyone and technical debt is introduced.

Needless Complexity

The design contains elements that are currently not useful. The added complexity makes the code harder to comprehend. Therefore, extending and changing the code results in higher effort than necessary.

Needless Repetition

Code contains exact code duplications or design duplicates (doing the same thing in a different way). Making a change to a duplicated piece of code is more expensive and more error-prone because the change has to be made in several places with the risk that one place is not changed accordingly.

Opacity

The code is hard to understand. Therefore, any change takes additional time to first reengineer the code and is more likely to result in defects due to not understanding the side effects.

Class Design

Single Responsibility Principle (SRP)

A class should have one, and only one, reason to change.

Open Closed Principle (OCP)

You should be able to extend a classes behaviour without modifying it.

Liskov Substitution Principle (LSP)

Derived classes must be substitutable for their base classes.

Dependency Inversion Principle (DIP)

Depend on abstractions, not on concretions.

Interface Segregation Principle (ISP)

Make fine grained interfaces that are client-specific.

Classes Should be Small

Smaller classes are easier to grasp. Classes should be smaller than about 100 lines of code. Otherwise, it is hard to spot how the class does its job and it probably does more than a single job.

Do stuff or know others, but not both

Classes should either do stuff (algorithm, read data, write data, ...) or orchestrate other classes. This reduces coupling and simplifies testing.

Package Cohesion

Release Reuse Equivalency Principle (RREP)

The granule of reuse is the granule of release.

Common Closure Principle (CCP)

Classes that change together are packaged together.

Common Reuse Principle (CRP)

Classes that are used together are packaged together.

Package Coupling

Acyclic Dependencies Principle (ADP)

The dependency graph of packages must have no cycles.

Stable Dependencies Principle (SDP)

Depend in the direction of stability.

Stable Abstractions Principle (SAP)

Abstraction increases with stability

General

Follow Standard Conventions

Coding-, architecture-, design guidelines (check them with tools)

Keep it Simple, Stupid (KISS)

Simpler is always better. Reduce complexity as much as possible.

Boy Scout Rule

Leave the campground cleaner than you found it.

Root Cause Analysis

Always look for the root cause of a problem. Otherwise, it will get you again.

Multiple Languages in One Source File

C#, Java, JavaScript, XML, HTML, XAML, English, German ...

Environment

Project Build Requires Only One Step

Check out and then build with a single command.

Executing Tests Requires Only One Step

Run all unit tests with a single command.

Source Control System

Always use a source control system.

Continuous Integration

Assure integrity with Continuous Integration

Override Safeties

Do not override warnings, errors, exception handling – they will catch you.

Dependency Injection

Decouple Construction from Runtime

Decoupling the construction phase completely from the runtime helps to simplify the runtime behaviour.

Design

Keep Configurable Data at High Levels

If you have a constant such as default or configuration value that is known and expected at a high level of abstraction, do not bury it in a low-level function. Expose it as an argument to the low-level function called from the high-level function.

Don't Be Arbitrary

Have a reason for the way you structure your code, and make sure that reason is communicated by the structure of the code. If a structure appears arbitrary, others will feel empowered to change it.

Be Precise

When you make a decision in your code, make sure you make it precisely. Know why you have made it and how you will deal with any exceptions.

Structure over Convention

Enforce design decisions with structure over convention. Naming conventions are good, but they are inferior to structures that force compliance.

Prefer Polymorphism To If/Else or Switch/Case

"ONE SWITCH": There may be no more than one switch statement for a given type of selection. The cases in that switch statement must create polymorphic objects that take the place of other such switch statements in the rest of the system.

Symmetry / Analogy

Favour symmetric designs (e.g. Load – Save) and designs that follow analogies (e.g. same design as found in .NET framework).

Separate Multi-Threading Code

Do not mix code that handles multi-threading aspects with the rest of the code. Separate them into different classes.

Misplaced Responsibility

Something put in the wrong place.

Code at Wrong Level of Abstraction

Functionality is at wrong level of abstraction, e.g. a PercentageFull property on a generic IStack<T>.

Fields Not Defining State

Fields holding data that does not belong to the state of the instance but are used to hold temporary data. Use local variables or extract to a class abstracting the performed action.

Over Configurability

Prevent configuration just for the sake of it – or because nobody can decide how it should be. Otherwise, this will result in overly complex, unstable systems.

Micro Layers

Do not add functionality on top, but simplify overall.

Dependencies

Make Logical Dependencies Physical

If one module depends upon another, that dependency should be physical, not just logical. Don't make assumptions.

Singletons / Service Locator

Use dependency injection. Singletons hide dependencies.

Base Classes Depending On Their Derivatives

Base classes should work with any derived class.

Too Much Information

Minimise interface to minimise coupling

Feature Envy

The methods of a class should be interested in the variables and functions of the class they belong to, and not the variables and functions of other classes. Using accessors and mutators of some other object to manipulate its data, is envying the scope of the other object.

Artificial Coupling

Things that don't depend upon each other should not be artificially coupled.

Hidden Temporal Coupling

If, for example, the order of some method calls is important, then make sure that they cannot be called in the wrong order.

Transitive Navigation

Aka Law of Demeter, writing shy code.

A module should know only its direct dependencies.

Naming

Choose Descriptive / Unambiguous Names

Names have to reflect what a variable, field, property stands for. Names have to be precise.

Choose Names at Appropriate Level of Abstraction

Choose names that reflect the level of abstraction of the class or method you are working in.

Name Interfaces After Functionality They Abstract

The name of an interface should be derived from its usage by the client.

Name Classes After How They Implement Interfaces

The name of a class should reflect how it fulfils the functionality provided by its interface(s), such as MemoryStream : IStream

Name Methods After What They Do

The name of a method should describe what is done, not how it is done.

Use Long Names for Long Scopes

fields → parameters → locals → loop variables
long → short

Names Describe Side Effects

Names have to reflect the entire functionality.

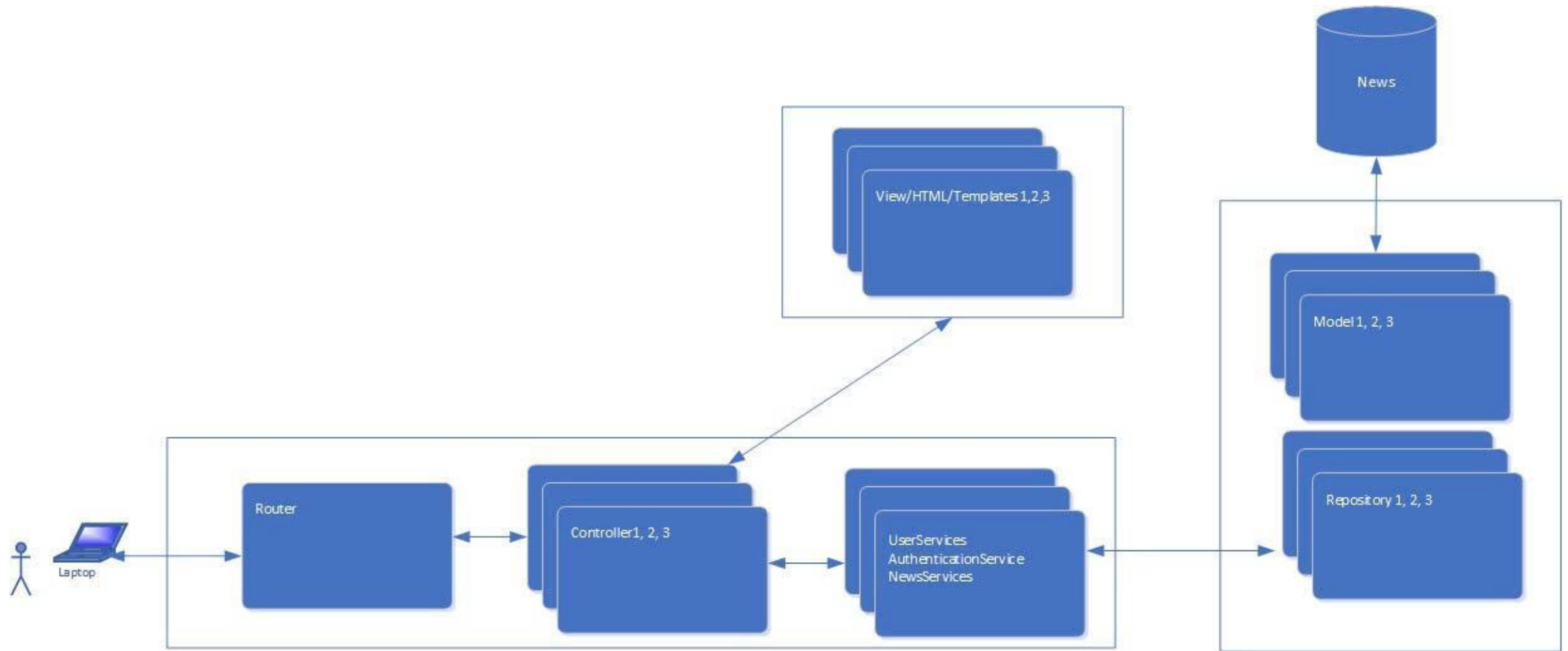
Standard Nomenclature Where Possible

Don't invent your own language when there is a standard.

Encodings in Names

No prefixes, no type/scope information

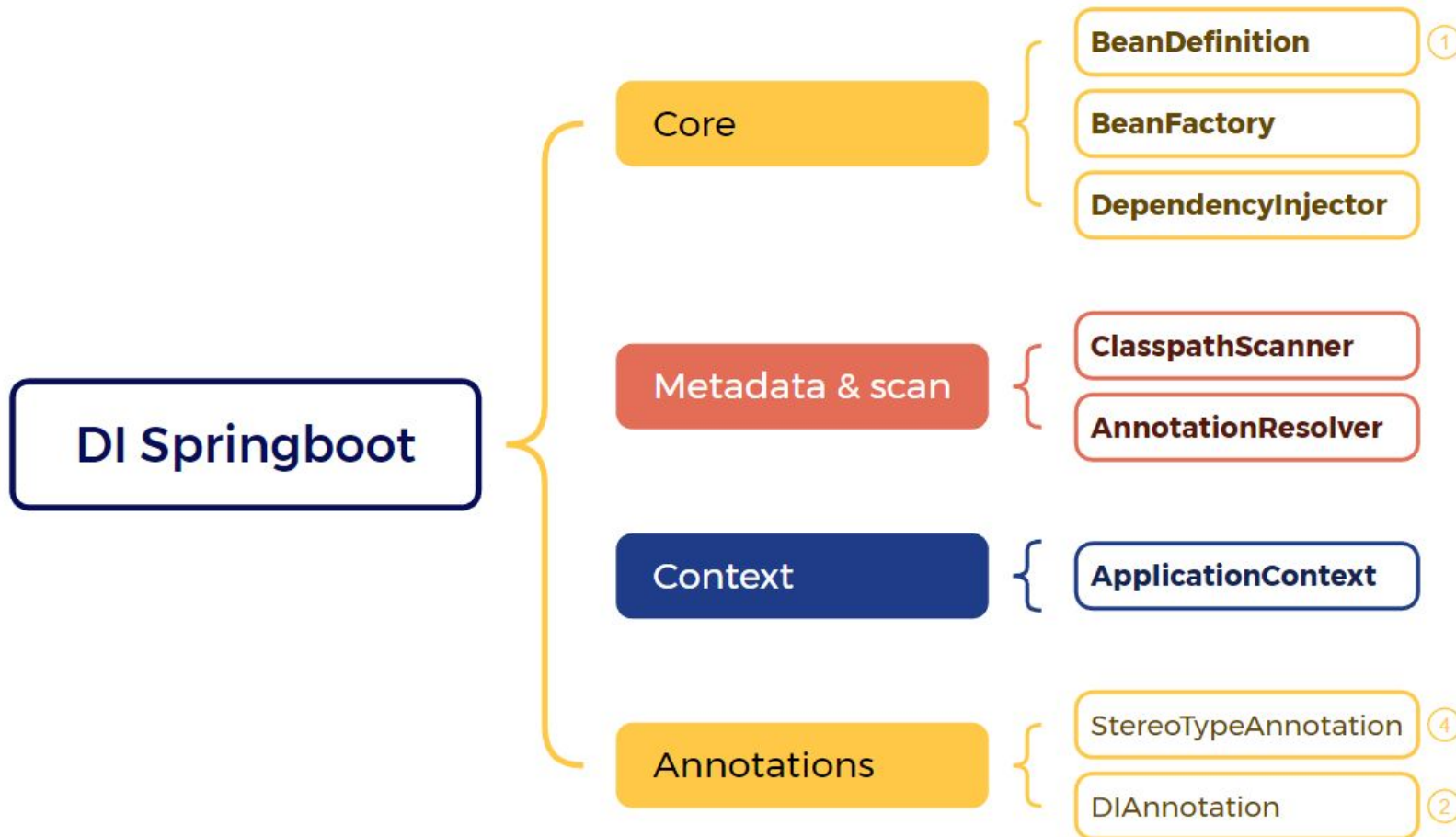
LEARN MVC MODEL





MLESTONE 4: WRITING CUSTOM FRAMEWORK & OUTDOOR ASSIGNMENT

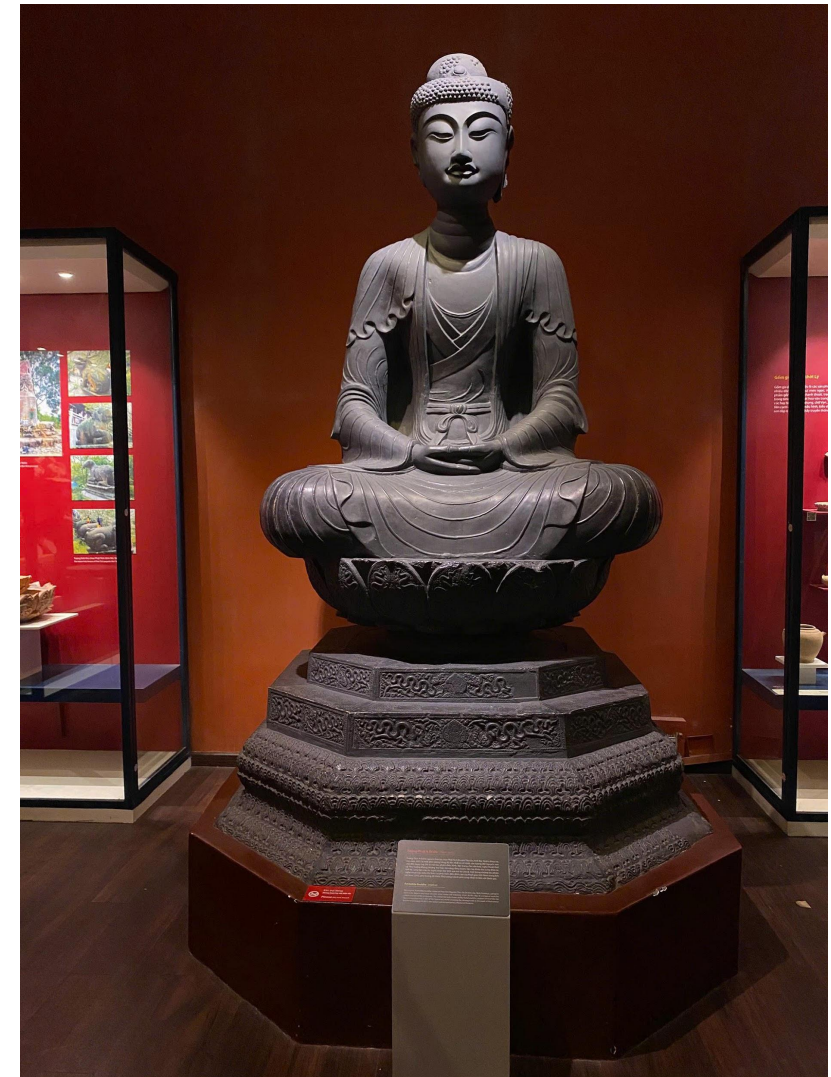
WRITING CUSTOM FRAMEWORK



TALKING WITH FOREIGNER



MAKE PRESENTATION ABOUT VIETNAM HISTORY AND CULTURE



A close-up photograph of a light-colored wooden block with the word "SUMMARY" printed in bold, black, sans-serif capital letters. The block is part of a larger arrangement of similar wooden blocks, some of which are visible in the background and foreground, creating a sense of depth. The lighting is soft and even, highlighting the natural grain of the wood.

SUMMARY

SUMMARY



- Learn from feedback and solving problem
- Learn both technical and soft skills
- Better everyday by daily progress

 phmquan Merge pull request #67 from phmquan/dev3 		95ca284 · yesterday	 153 Commits
 SQL_Query	add TestTemplate and test Template Category	last week	
 Unit_Test	Implement custom MVC Framework	yesterday	
 simulate_sql_query/src	update SQL Query Pivot Card CustomField	2 weeks ago	
 README.md	Create README.md	last month	



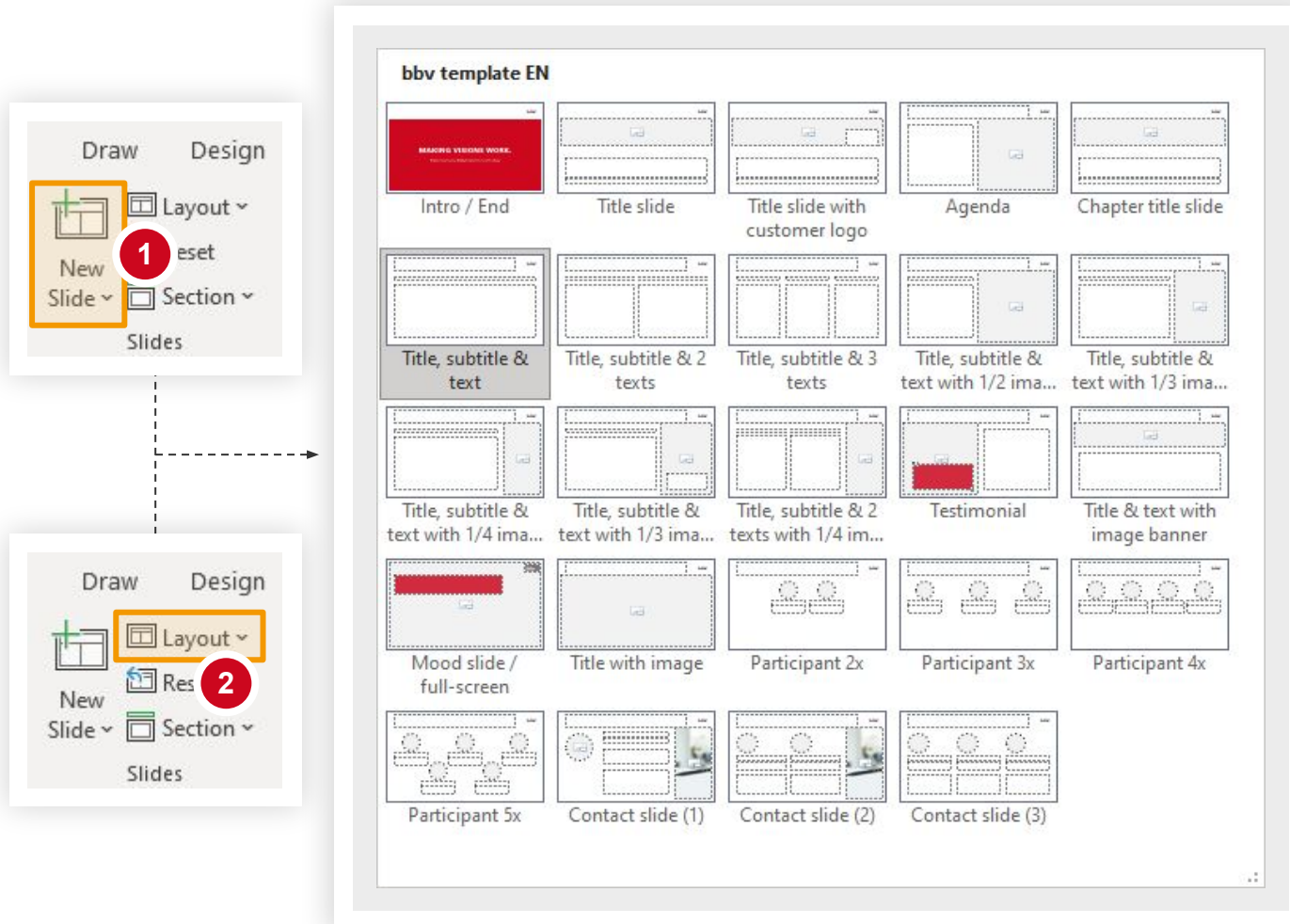
MAKING VISIONS WORK.

Co-Creating Digital Businesses, Digital Experiences & Products

How-to guides

bbv Master layouts & slide library

Insert new slide



1

Click on «New Slide»

2

Select layout

Choose the desired layout from the pre-programmed master layouts.



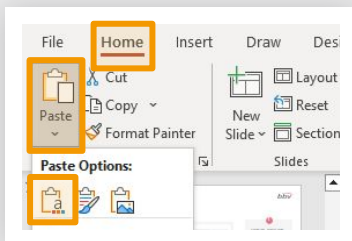
Or select layout from slide library

Transfer of existing slides into the new Master

1

Insert original slide

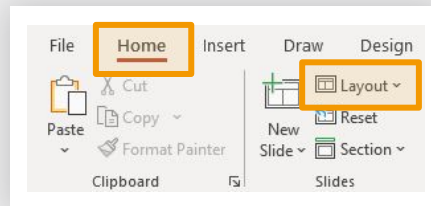
1. Copy slide in original presentation with Ctrl+C.
2. In the target presentation, select the slide after which the original slide is to be inserted.
3. Select in the tab «Home / Paste / Use Destination Theme».



2

Assign slide layout

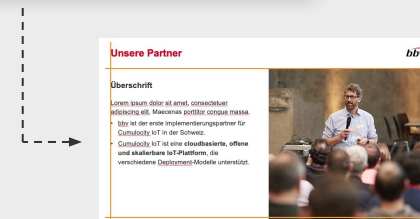
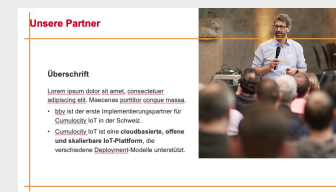
1. Select the inserted slide
2. Select the desired layout in the tab «Home / Layout».



3

Adjust slide manually

- Check all slide contents such as text blocks, images, etc. for position and size and manually adjust them to the new layout grid.



4

Check design

Independently check:

- Font type, sizes and font colours
- Colours of graphic elements
- Text alignment
- Coherence of the overall picture

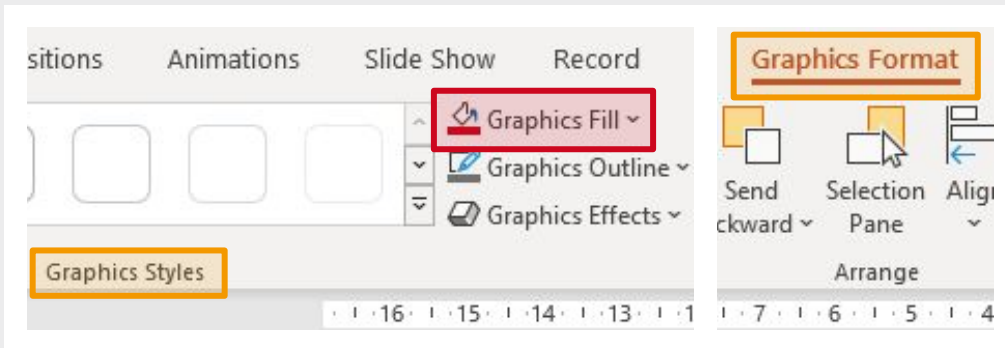
Important:

When transferring existing slides, always carry out all four steps.

Change icon colour

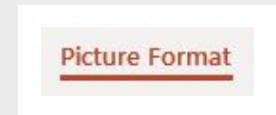
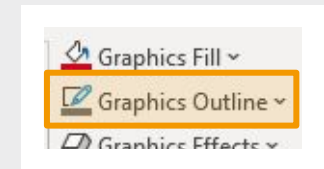
Approach

1. Select icon
2. In the tab «Graphics Format» under «Graphics Styles», use «**Graphics Fill**» to change the colour

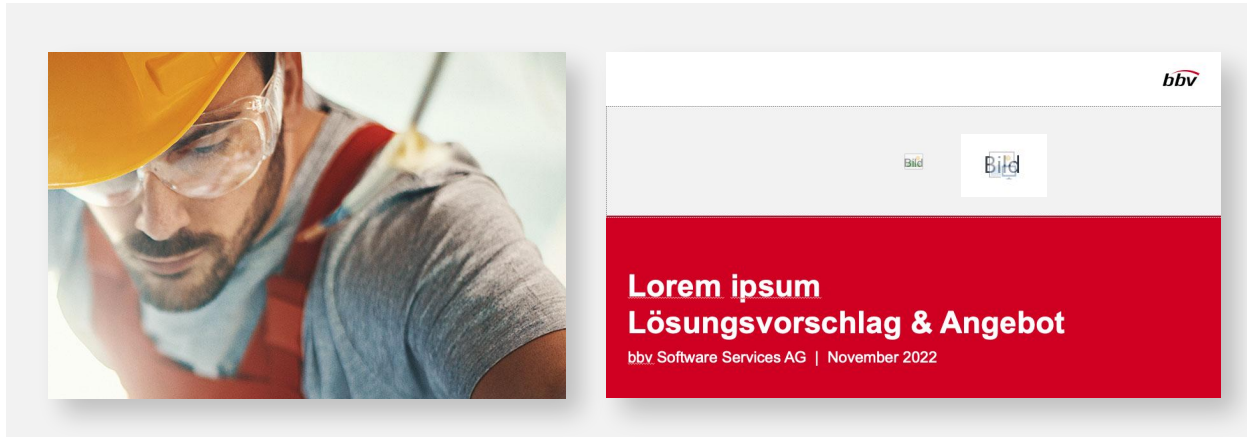


Notes

- For some icons, the colour can be changed by using «**Graphics Outline**» instead of «Graphics Fill».
- The colour cannot be changed if an icon is inserted as a picture. In this case, after selecting the icon, the tab «**Picture Format**» appears instead of «Graphics Format».
- Best vector file format for icons: *.svg

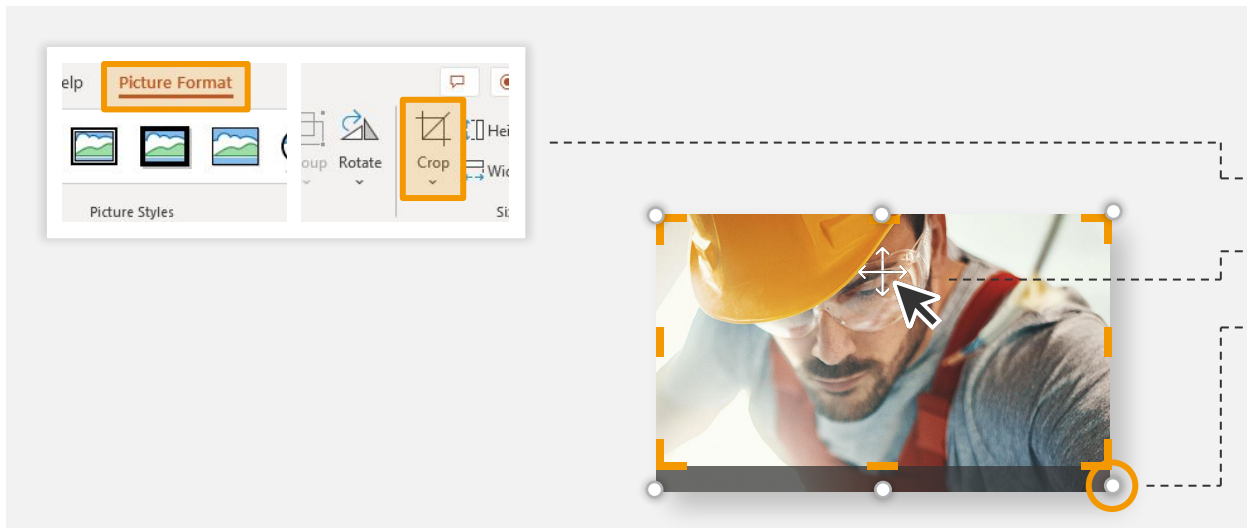


Insert image into placeholder



Insert image from external source

- Click on the image  icon, select image from Sharepoint or other source
- The bbv image library is here locally located:
C:\Users\Public\bbv_templates\Marketing\PPT-Bibliotheken\Bilder-Bibliothek
- Or: copy image from external source with Ctrl+C, select the placeholder, then paste with Ctrl+V



Adjust

1. Select the image
2. Click in the tab «Picture Format» on «Crop»
3. Adjust the image composition by moving the image
4. Resize the image by clicking and dragging the outer circles.
5. Click again on «Crop» to finish the adjustments

Insert image into mock-up screen (mobile/laptop)



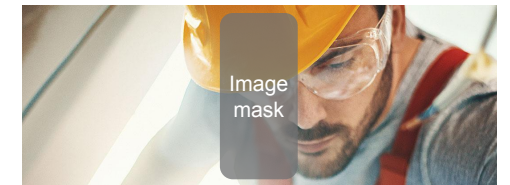
Paste image

- Copy image from external source with Ctrl+C and paste with Ctrl+V



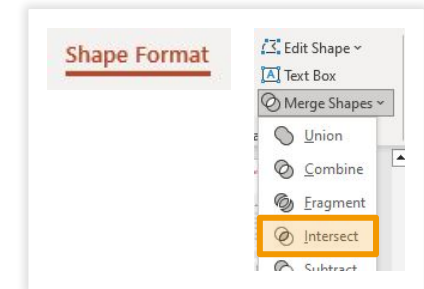
Adjust size & determine image area:

1. Place image behind image mask
2. Adjust image size and determine image area



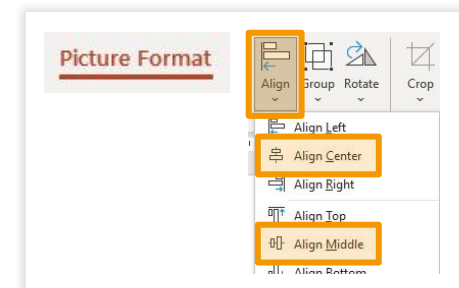
Crop image to image mask

1. Select image
2. Select image mask
3. Click in the tab «Shape Format / Merge Shapes» on «Intersect»



Combine image with mock-up

1. Place image behind mock-up
2. Activate image and mock-up and align both horizontally (Align Center) and vertically (Align Middle)
3. Group image and mock-up with Ctrl+G



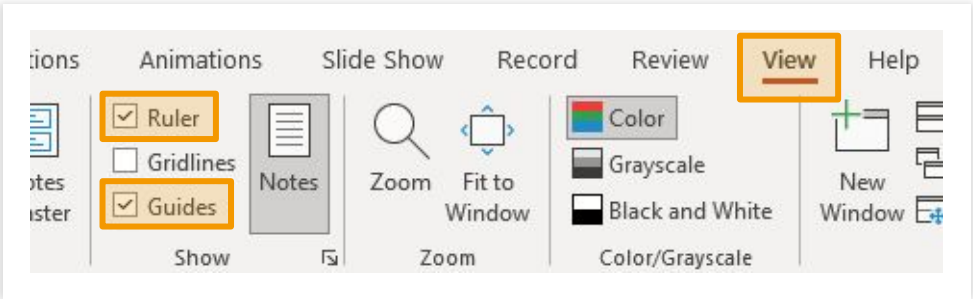
Style guide

Working area, guides & ruler

Working area

Depending on the slide, a single-column layout, a 2-column or 3-column layout is used.

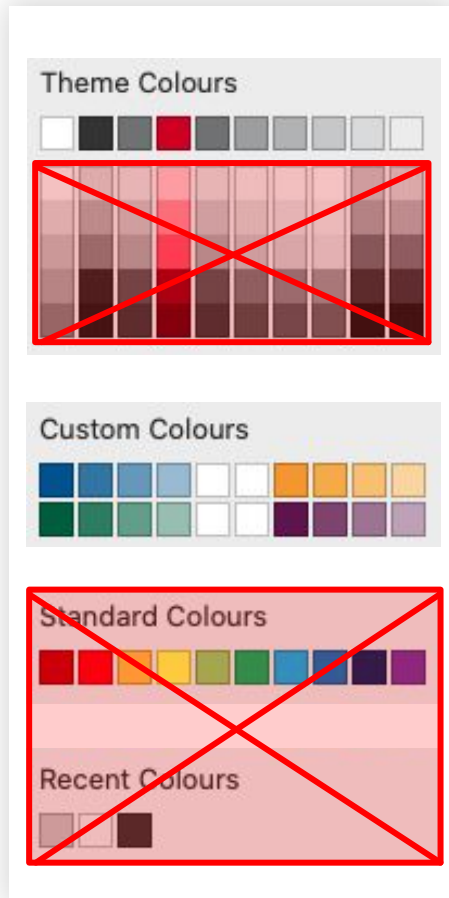
Display the ruler and guides in the «View» tab.



Agenda		bbv	
1.	Begrüßung & Vorstellung der bbv Vorname Nachname		
2.	Lösungsvorschlag Vorname Nachname		
3.	Vorgehen, Methode und Termine Vorname Nachname		
4.	Zusammenarbeitsmodell Vorname Nachname		
5.	Angebot Vorname Nachname		
6.	Vorzüge Angebot inkl. Referenzen Vorname Nachname		

Lorem ipsum		bbv	
Lorem ipsum	Lorem ipsum		
• Erste Ebene Bullet.	• Erste Ebene Bullet.		
• Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus.	• Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus.		
• Zweite Ebene Bullet eingezogen.	• Zweite Ebene Bullet eingezogen.		
• Nullam dictum felis eu pede mollis pretium. Integer tincidunt.	• Nullam dictum felis eu pede mollis pretium. Integer tincidunt.		

bbv colour theme



- Always OK!
- Never, please!
- Use selectively and sparingly
- Never, please!

